Operating Systems & Systems Architecture

Autonomous Comprehensive Synthesis • Authored by Cognexa Autonomous AI

Subject:	Operating Systems & Systems Architecture	**Date:**	Current Operational Cycle
Engine:	Cognexa Core 2.0 (LangGraph + Vector Memory)	**Security:**	Verified & Sandboxed

1. Executive Summary

An Operating System (OS) is the fundamental system software that manages computer hardware, system resources, and provides common services for application programs. Key functions include process management, virtual memory virtualization, filesystem abstractions, and I/O scheduling.

2. Process & Thread Management

The kernel encapsulates executing code within isolated process memory boundaries.

- Process Control Block (PCB): Process states (New, Ready, Running, Waiting, Terminated), register state, memory maps, and open file descriptors.
- Threads vs Processes: Threads share the same address space and heap but maintain independent stacks and program counters; Kernel threads vs User-level green threads.
- CPU Scheduling Algorithms: Preemptive vs Non-preemptive scheduling; Round Robin, Multi-Level Feedback Queues (MLFQ), and Completely Fair Scheduler (CFS).
- Inter-Process Communication (IPC): Unix Domain Sockets, Pipes, Shared Memory with synchronization barriers, and Message Queues.

3. Concurrency, Synchronization & Deadlocks

Concurrent execution requires deterministic synchronization primitives to prevent race conditions.

- Critical Section Problem: Mutual exclusion, progress, and bounded waiting requirements; Peterson algorithm and hardware Test-and-Set / Compare-and-Swap (CAS) instructions.
- Synchronization Primitives: Mutexes, Counting/Binary Semaphores, Spinlocks, Reader-Writer locks, and Condition Variables.
- The Deadlock Conditions: Coffman conditions (Mutual Exclusion, Hold and Wait, No Preemption, Circular Wait).
- Deadlock Handling Strategies: Prevention, Avoidance via Banker algorithm, Detection using resource allocation graphs, and recovery via process preemption.

4. Virtual Memory Management & File Systems

Virtual memory provides each process with the illusion of a vast, contiguous address space.

- Paging & Address Translation: Memory Management Unit (MMU) translating virtual addresses to physical frames using page tables; Multi-level page tables and Translation Lookaside Buffer (TLB).
- Page Faults & Replacement: Handling missing frames; Page replacement algorithms: Least Recently Used (LRU), Clock algorithm, and Second Chance.
- File System Architecture: Inodes storing metadata and data block pointers; Virtual File System (VFS) abstraction; Journaling (ext4, XFS) preventing filesystem corruption.
- I/O Management: Direct Memory Access (DMA) offloading CPU interrupts; Disk scheduling algorithms; Block caching in system memory.